# ANVAYA AI

Complete Project Specification & Engineering Blueprint

Multilingual, voice-first welfare scheme discovery, eligibility pre-screening, document guidance and application-draft assistance

**Version 1.0 · 18 October 2026**

Consolidated from the project brief, supplied screenshots, prior planning discussion, official workflow/test-case notes, the hybrid edge-cloud proposal, and the supplied scheme-data ZIP.

**Tagline direction:** “Every voice, a way forward.”

**Document status:** implementation blueprint. It describes intended architecture and planned behavior; it does not assert that cloud deployment, GraphDB, offline sync, government integrations, or document verification have already been built.

# How to read this specification

This document deliberately separates (A) the assigned problem and organizer workflow, (B) the proposed hackathon MVP, (C) originality features to demonstrate, and (D) production/advanced concepts. This avoids promising capabilities that a prototype cannot substantiate.

- **Assigned problem:** a citizen asks in Telugu/Hindi which government schemes their household may qualify for; the system collects household details, consults official/read-only or mock scheme sources, applies current rules provisionally, pre-fills an application for review, and returns a voice summary and status.
- **Core engineering principle:** the LLM is a conversational/semantic interface, not the authority on eligibility. Structured facts feed a deterministic, versioned rules evaluator. Unknown or unverified facts stay unknown.
- **Architecture commitment from your screenshot:** hybrid edge-cloud kiosk: locally cached core scheme data and SQLite rules for offline preliminary screening; cloud performs heavier document OCR, wider data cross-reference, and PDF generation, with queued synchronization when connectivity returns.
- **Knowledge Graph decision:** model scheme/criteria/document/state/department relationships as a graph. For the MVP, keep deterministic rules as the decision authority; a graph database such as Neo4j is optional until graph traversal demonstrably helps. A graph-shaped data model can be implemented first in PostgreSQL/SQLite.
- **Demo truthfulness:** the dataset is an input catalogue, not proof that all entries are current/official. Show source URL, imported/verified timestamps, confidence/review status, and provisional outcomes.

# Contents


- 1. Vision, users, scope and differentiation
- 2. Official workflow, rubric and test cases
- 3. Product journeys and UX
- 4. Dataset, Knowledge Graph and eligibility model
- 5. Hybrid edge-cloud HLD
- 6. Technology stack and deployment
- 7. Functional and non-functional requirements
- 8. LLD: modules, schemas, rule outcomes and state machine
- 9. REST API and frontend routes
- 10. Offline sync, security, freshness and audit
- 11. Test plan, judge demo and implementation roadmap
- 12. Advanced ideas: adopt, defer or reject
- 13. Assumptions and open decisions

# 1. Vision, users, scope and differentiation

**Product:** Anvaya AI helps people discover potentially relevant government welfare schemes, understand eligibility and required documents in a familiar language, and prepare an application draft for human review. It is designed for multilingual, low-literacy and assisted-service contexts, including rural/semi-urban kiosks with unreliable connectivity.

## Primary users

- **Citizen:** asks by voice or text; confirms facts; sees scheme matches, reasons, document gaps, next steps and draft.
- **Kiosk/assisted operator:** starts a temporary session and assists without receiving unrestricted access to sensitive data.
- **Authorized reviewer:** sees consented drafts, checks evidence, requests corrections or records a review outcome.
- **Scheme-data steward/admin:** imports, validates and reviews catalogue/rule updates.

## Product surfaces

- One responsive citizen web app with a **Personal** experience and a **Kiosk/Assisted** mode toggle; same core backend.
- Lean role-protected official-review dashboard.
- Local kiosk runtime/cache for offline preliminary screening; cloud services for sync and heavy processing.
- Personal account/identity federation, real government submission, real DigiLocker, live CSC integrations and full offline AI are not MVP promises.

## Why it can stand out

- **End-to-end last-mile completion:** not just scheme Q&A;—collect confirmed facts, match schemes, explain document gaps, draft, and hand off to a reviewer.
- **Trustworthy decisions:** explicit rule comparisons, reason traces, citations, provisional/unknown statuses and human review.
- **Offline-resilient kiosk:** basic screening continues during connectivity loss; sync and heavier work resume later.
- **Voice correction loop:** deliberately surface and correct misheard age/income/location, then recompute and show what changed.
- **Judge-visible reproducibility:** TC1–TC4 buttons run seeded cases and expose input, expected/actual output and evidence.

## Keep the winning scope tight

Build one polished citizen journey, the kiosk configuration, an official review queue, deterministic rules, and a small offline cache/sync demonstration. Do not build three independent apps or claim production identity/security. Prioritize proof of reliability over a broad but fragile feature list.

# 2. Official workflow, judging and test cases

The supplied project workflow describes seven evidence-oriented steps. Treat these as the acceptance backbone, not optional UX suggestions.

| Step | What the system must do | Evidence in demo |
|---|---|---|
| 1. Inputs | Collect language, district/state, household members, income, occupation and other required attributes; give facts stable IDs and validate them. | Profile summary with confirmed/unconfirmed fields and input IDs. |
| 2. Named sources | Use official scheme portals or organizer-provided mock/read-only sources; preserve source IDs/URLs and retrieval times. | Scheme source card, source URL, retrieved/imported/verified metadata. |
| 3. Decision rules | Explicit comparisons; provisional label; save input evidence/reason; unknown if key evidence is missing. | Rule-by-rule explanation, outcome enum, reason array, rule version. |
| 4. Permitted action | Pre-fill application draft for review; show proposed data/action; require citizen approval; make retries idempotent. | Draft preview, consent/approval event, same draft returned on retry. |
| 5. Check actual result | Read application/case status separately from draft/action; keep pending/failed visible. | Status badge and timestamp; no fabricated submitted/approved status. |
| 6. Result view | Voice summary in user language; audio/visual status; link decision back to input record. | Summary plus traceability to input IDs and scheme sources. |
| 7. Run scenarios | Save/run four labelled official test cases with input, expected result, actual result and evidence. | In-app TC1–TC4 runner with pass/fail and reset. |

## Official test cases (TC1–TC4)

| ID | Scenario | Expected behavior | Implementation proof |
|---|---|---|---|
| TC1 | Eligible household | Show candidate scheme list with relevant rules quoted. | Rule snippets, source citations, deterministic criterion trace. |
| TC2 | Same/similar scheme name in two states | Correct state/jurisdiction is used; central schemes may also be considered. | Run identical query with State A then State B; show different scoped records/source. |
| TC3 | Age misheard | Confirm critical age before filtering; accept correction and recalculate. | Show transcript/extracted age, confirmation, corrected value and changed results. |
| TC4 | Provisional eligibility | Never promise benefit when proof is missing/unverified. | PROVISIONAL or INSUFFICIENT_INFORMATION, document gap and saved draft. |

## Additional screenshot-derived checks

- Decision response should include structured `decision_reason[]`, criterion outcomes, `source_id`, `source_url`, `rule_version` and input fact IDs.
- Handoff/save is idempotent: repeating the same session/code updates or returns the existing draft, not a duplicate.
- Dashboard shows explicit states such as `DRAFT_SAVED`, `PENDING_REVIEW`, `CORRECTION_REQUESTED`, `REVIEWED`; `SUBMITTED_EXTERNAL` is reserved for a real verified integration.
- A dedicated test-runner route/button bar should launch TC1, TC2, TC3 and TC4 without judges manually recreating each input.
- Ask human approval before an action that cannot easily be undone; use synthetic/mock data for sensitive demo workflows; do not bypass OTP/CAPTCHA.

## Rubric note

The screenshots include two rubric framings: one review overview (problem fit/usefulness, agentic capability, technical implementation, innovation, UX, presentation) and one hackathon scorecard (working functionality, technical integration, multilingual performance, engineering originality, safety/reliability, reproducibility). Use the organizer's currently authoritative

rubric; design the demo evidence to satisfy both framings rather than assuming one replaces the other.

# 3. User journeys and interaction design

## Citizen journey

- Choose supported language; start voice or text. VerbX APIs handle speech recognition and speech output; do not train/build ASR or TTS from scratch for the hackathon.
- Ask for relevant household facts in short turns: state/district, age, household members, income, occupation, category/beneficiary attributes only when needed.
- Show a compact profile confirmation card. Confirm high-impact speech-derived facts (age, income, location, household composition) before rules run. Citizen can correct by voice/text.
- Run jurisdiction-aware candidate retrieval, then deterministic criteria evaluation.
- Show a Scheme Passport: potential match / mismatch / insufficient information, why, source, rule excerpt, missing/unverified proof, benefit/next step—without implying guaranteed entitlement.
- Open Document Guide: required papers, plain-language purpose, citizen-marked available/missing/expired/unknown. A checkbox is not official verification.
- Pre-fill draft from confirmed facts; mark every field as confirmed, inferred, missing or citizen-entered. Citizen reviews and consents to handoff.
- Generate resumable session/case code. Official reviewer can inspect authorized/consented draft, request correction or record review. Return status in citizen's language.

## Kiosk/assisted mode

- Large touch targets, high contrast, minimal text, voice prompts, audio/visual state indicators and keyboard/accessibility support.
- Operator authenticates with a seeded demo staff account; starts temporary citizen session. Session code/QR is a locator, not proof of identity and must expire/rate-limit.
- On exit, clear citizen-sensitive details from shared screen; persist only consented case data under defined retention.
- Personal and kiosk modes share the same journey/backend; kiosk is a responsive/accessibility configuration, not a separate product.

## Official reviewer journey

- Seeded demo reviewer login; role checked server-side.
- Queue filters by status; view only cases authorized for the reviewer.
- Inspect profile facts, source evidence, rule traces and document gaps; request correction or record review outcome.
- Log meaningful actions with actor, timestamp, case, action and version. Do not imply real government authority.

## Multilingual continuity

Allow language switching mid-session without losing structured profile or case state. Keep canonical values (e.g., normalized state and numeric income) separate from localized display strings. Test Hindi/Telugu and any additional language only if VerbX support and quality are actually verified.

# 4. Dataset, Knowledge Graph and eligibility model

## Dataset supplied

The supplied ZIP was previously inspected as containing `structured_govt_schemes_converted.csv` (reported 3,534 rows, 3,534 distinct scheme names, 37 state/territory values). Confirm these counts against the exact extracted file at implementation time. Column families include scheme identity/name, state, department, level, categories, beneficiaries, description/benefits, eligibility text, application process, required documents, source URLs and FAQs. A row count is not proof of freshness, official status or parseable eligibility.

## Ingestion and governance

- Preserve original ZIP/CSV, checksum, filename and import timestamp as immutable source artifact.
- Normalize encoding, whitespace, HTML entities, state/UT aliases and list-like fields; preserve original raw fields alongside normalized fields.
- Validate duplicate IDs/names, missing/invalid URLs, malformed eligibility/documents, conflicting state/level values and blank critical fields.
- Attach `source_url`, `source_id`, `retrieved_at`, `imported_at`, `verified_at`, `review_status`, `confidence`, `rule_version` and optional `valid_from/valid_to`.
- Convert free-text eligibility into machine rules only when confidently interpretable and human-reviewed. Ambiguous rules remain `UNKNOWN/NEEDS_REVIEW`; do not let an LLM silently invent thresholds.
- Curate a small manually reviewed golden set for the demo; show the rest as catalogue entries with verification caveat.

## Knowledge Graph conceptual model

Welfare policies are relational: a scheme is scoped to a jurisdiction, targets beneficiary groups, has eligibility criteria, requires documents, offers benefits and is administered by a department. Model these relationships as a Knowledge Graph, but keep rule evaluation explicit and deterministic.

| Node types | Relationships (examples) | Useful query |
|---|---|---|
| CitizenProfile, Scheme, State/UT, Department, BeneficiaryGroup, Criterion, Document, Benefit, Source, RuleVersion | Profile—LIVES_IN→State; Scheme—APPLIES_IN→State; Scheme—ADMINISTERED_BY→Department; Scheme—TARGETS→BeneficiaryGroup; Scheme—HAS_CRITERION→Criterion; Scheme—REQUIRES→Document; Scheme—OFFERS→Benefit; Rule—SUPPORTED_BY→Source | Find central + applicable state schemes; traverse requirements; explain scheme-to-document/criterion path. |

## Storage decision

- **MVP recommended:** PostgreSQL for cloud relational records (or SQLite for a single-node prototype); represent graph edges with join tables and stable IDs. A graph-shaped schema is enough to demonstrate semantic relationships.
- **Optional GraphDB/Neo4j:** introduce if the team can show graph traversal materially improves multi-hop discovery or relationship explanations. Import nodes/edges from the validated catalogue; preserve rule engine as the authority.
- **Offline kiosk:** ship a compact, signed/versioned subset of scheme records and compiled deterministic rules in SQLite. Do not require a live cloud graph query for basic pre-screening.
- **Vector search:** optional for fuzzy scheme-description retrieval, not for eligibility truth. Use it only as candidate retrieval; filter jurisdiction and run explicit rules afterward.

## Rule outcome contract

Suggested outcomes: `MATCH` (all required conditions supported by confirmed facts and required evidence state is satisfied), `PROVISIONAL` (criteria appear satisfied but proof/verification is missing or source is not current), `MISMATCH` (a confirmed

criterion fails), `INSUFFICIENT_INFORMATION` (critical profile fact/rule is unknown), `NEEDS_HUMAN_REVIEW` (rule text/source ambiguity). Each outcome includes criterion-level results, reason list, input fact IDs, rule version and citations.

# 5. Hybrid edge-cloud high-level architecture

This section explicitly incorporates the supplied “Edge-Optimized, Offline-Resilient Kiosk Architecture” proposal.

## Logical architecture

```
CITIZEN / ASSISTED KIOSK (EDGE)
  Responsive PWA / kiosk UI
  ■■ Voice/text capture; VerbX cloud STT/TTS when online
  ■■ Local session + consent state
  ■■ SQLite: compact scheme catalogue, jurisdiction map,
  ■          reviewed rules, source/version metadata
  ■■ Local deterministic pre-screening (offline-capable)
  ■■ Encrypted/controlled outbox: drafts + sync events
  ■■ Offline status + stale-data warning
                ■ secure sync when network returns
                ▼
CLOUD / CENTRAL SERVICES
  API gateway / HTTPS
  ■■ Flask application API + session orchestration
  ■■ Scheme catalogue + versioned rule service
  ■■ Knowledge Graph layer (relational graph MVP;
  ■                         optional Neo4j/GraphDB)
  ■■ VerbX integration adapter (STT/TTS/translation)
  ■■ Document service (optional OCR / extraction)
  ■■ Draft/case/reviewer service + audit
  ■■ PDF generation / heavy cross-reference jobs
  ■■ PostgreSQL (production direction; SQLite POC)
  ■■ Object storage for consented documents (optional)
  ■■ Worker/cron: curated data refresh, sync processing,
                 expiry cleanup, retry/backoff
```

## Edge responsibilities

- Run basic eligibility pre-screening locally using cached core scheme data and deterministic SQLite rules; it should work without a network for the explicitly cached scope.
- Display cache version and last successful refresh. Offline outcomes are preliminary and can be stale; no final approval/submission claim.
- Queue draft/sync events locally with stable event IDs and idempotency keys. Do not queue unnecessary raw audio by default.
- When offline, use typed input or locally available UI. VerbX cloud speech APIs are not assumed to work offline; if speech API cannot be reached, provide text fallback.

## Cloud responsibilities

- Authoritative shared catalogue/rule versions, multi-kiosk synchronization, reviewer dashboard, role checks and audit.
- Heavy OCR/document parsing, broader source cross-reference and complex PDF generation may run asynchronously after connectivity returns.
- Cloud can deliver signed/versioned catalogue bundles and rule updates to kiosks after validation.
- Cloud response and case status must distinguish queued/local draft, synced draft, pending review and any genuine external submission.

## Sync and conflict policy

- Each offline action has `device_id`, `session_id`, `event_id`, `created_at`, `catalogue_version`, `rule_version`, `idempotency_key` and payload schema version.
- At reconnect, upload queued events with retries/backoff; server deduplicates by event/idempotency key and returns canonical case/draft ID plus accepted/rejected/conflict status.

- Do not silently overwrite conflicting citizen/reviewer edits. Use draft version/optimistic concurrency; return a conflict requiring reconciliation.
- If the scheme/rule bundle changed while offline, retain the result as computed under its old version and re-evaluate online before handoff; tell the user if result changed.
- Local cache refresh is atomic: download/validate new bundle, then switch active version; retain last-known-good bundle if refresh fails.

## Practical MVP slice

Demonstrate offline behavior with a small curated, reviewed local bundle and SQLite rules; disconnect network, run a basic profile/scheme pre-screen, save a draft to an outbox, reconnect, sync it, and show the reviewer queue receives exactly one draft. Avoid claiming zero latency universally or full offline OCR/LLM.

# 6. Technology stack and deployment

| Layer | Hackathon recommendation | Why / boundaries |
|---|---|---|
| Frontend | React + Vite + TypeScript; Tailwind or equivalent; responsive PWA/kiosk UI | One app for citizen/kiosk and reviewer surfaces; accessibility, clear state badges, TC runner. |
| Backend/API | Python Flask (extend supplied starter if it exists); REST/JSON; schema validation | Simple, demoable service; don't rewrite a provided Flask codebase without reason. |
| Voice | VerbX supplied STT/TTS/translation endpoints via adapter | Real API calls where required; keep transcript correction and text fallback. Credentials via env secrets. |
| LLM | LLM for intent, entity/fact extraction, natural-language explanations/translation support | Never use LLM output as the sole eligibility verdict; validate extracted JSON and confirm high-impact facts. |
| Eligibility | Python deterministic rule evaluator; versioned rule JSON/DB rows | Criterion-level trace, reproducible results, provisional/unknown states. |
| Catalogue DB | SQLite for local POC/kiosk bundle; PostgreSQL for shared cloud deployment | Keep data model portable; choose one primary shared DB for MVP. |
| Knowledge graph | Graph-shaped relational schema first; Neo4j/GraphDB optional extension | Graph DB is not mandatory. Use if real multi-hop graph queries justify ops complexity. |
| Local edge | SQLite cache + PWA/local kiosk runtime + durable outbox | Offline pre-screening only for cached reviewed schemes; data encryption/retention must be designed. |
| Async jobs | Simple worker/cron (Flask CLI/Celery/RQ only if needed) | Dataset validation/refresh, retry sync, PDF/OCR jobs, expired session cleanup. |
| Files/OCR | Optional cloud object storage + OCR service; local file-size/type validation | Demo with synthetic documents. OCR extracts candidates; does not authenticate documents. |
| Deployment | Docker Compose for reproducible demo; cloud VM/container or managed app platform for online API; HTTPS | Use a concrete provider only after confirming credentials/credits and deployment target; do not claim deployed cloud infra until tested. |
| Observability | Structured logs, request IDs, health/readiness endpoints, basic metrics | Avoid PII/raw audio in logs; capture trace IDs for judge evidence. |

## Cloud provider choice

The supplied screenshot specifies a hybrid edge-cloud pattern but does not, by itself, mandate AWS/Azure/GCP. Provider-specific services should be selected based on hackathon access, credits and team familiarity. Provider-neutral logical components above can be mapped later. State exactly which components run locally, in the cloud, or are mocked in the demo.

## Deployment profiles

- **Dev/demo:** Docker Compose, Flask API, SQLite/seed fixtures, frontend, optional worker; local kiosk simulation and test-runner.
- **Cloud demo:** HTTPS-hosted frontend/API, managed PostgreSQL if available, object storage only if document uploads are enabled, worker for background jobs.
- **Production direction:** identity provider, least-privilege access, managed DB/backups, secrets manager, monitoring, encryption, approved integrations, data retention and policy governance. Requires security/privacy review.

# 7. Functional requirements (FR)

| ID | Feature | Requirement | Acceptance check |
|---|---|---|---|
| FR-01 | Language and input | Select supported language; voice via VerbX or text; graceful fallback. | Text remains usable on voice/API outage; language switch preserves state. |
| FR-02 | Profile extraction | Extract age, state/district, income, occupation, household attributes; provenance per fact. | High-impact facts confirmed before screening; corrections persisted. |
| FR-03 | Clarification | Ask targeted questions for missing/ambiguous required attributes. | No repeated questions for confirmed facts; unknowns remain unknown. |
| FR-04 | Scheme catalogue | Search/filter by jurisdiction, level, category, beneficiary, keyword. | Each displayed record has source and freshness/review label. |
| FR-05 | Rule evaluation | Deterministic, versioned criteria evaluator; return criterion-level outcomes. | Same facts + rule version => same result; unknown facts not guessed. |
| FR-06 | Scheme Passport | Multiple matches with reasons, rule snippets, benefits, gaps, next steps, citations. | Clearly distinguishes match/provisional/mismatch/insufficient info. |
| FR-07 | Document Guide | Checklist; plain-language explanation; citizen marks available/missing/unknown. | No “verified” label without actual verification. |
| FR-08 | Draft preparation | Pre-fill from confirmed facts; show missing/unconfirmed fields; edit/consent. | Draft field provenance; citizen approves before handoff. |
| FR-09 | Kiosk session/handoff | Temporary expiring code/QR; resume permitted draft. | Code alone cannot expose sensitive case; repeated handoff idempotent. |
| FR-10 | Reviewer workflow | Authorized queue; inspect, request correction, record review. | Server role checks; audit meaningful actions. |
| FR-11 | Status truthfulness | Separate saved draft, synced, pending review and external submission. | No submitted/approved claims absent verifiable integration result. |
| FR-12 | Freshness | Import/verification timestamps; stale/unreviewed warning. | Old/uncertain rules not represented as current verified policy. |
| FR-13 | Test runner | Run TC1–TC4 with preloaded inputs and expected outputs. | Judge sees input, expected, actual, pass/fail and trace. |
| FR-14 | Consent/audit | Consent before sharing; trace profile confirmation, evaluation, draft, review. | Data-minimized audit trail, stable input IDs. |
| FR-15 | Offline screening | Run local subset of reviewed rules and cached schemes without connectivity. | UI marks offline/preliminary, shows cache/rule version and timestamp. |
| FR-16 | Sync/reconciliation | Queue offline draft/events; retry and reconcile when online. | Deduplication, visible sync state, conflicts surfaced. |
| FR-17 | Document pre-check (stretch) | Optional OCR extracts candidate fields/date/issuer from synthetic/demo documents. | Human confirms extracted values; no authenticity guarantee. |
| FR-18 | Dialect glossary (stretch) | Map colloquial/local administrative terms to canonical scheme taxonomy. | Tested examples; preserve original utterance and ask clarification if uncertain. |

# 8. Non-functional requirements (NFR)

| Area | Requirement / measurable expectation |
|---|---|
| Correctness | Eligibility rules deterministic, unit-tested, independent of generated prose; maintain gold fixtures. |
| Reliability | Recoverable API failures; text fallback; retry-safe/idempotent draft and sync; pending/failure visible. |
| Performance | Measure separately: local rules, cloud API, VerbX latency, OCR/PDF. Target sub-second local rule evaluation on demo-sized cache, not end-to-end voice. |
| Accessibility | Keyboard support, high contrast, large kiosk controls, simple language, audio/visual status, screen-reader labels. |
| Privacy | Minimize profile/audio/document collection; consent; retention policy; shared-device cleanup; avoid unnecessary raw audio. |
| Security | HTTPS, server-side validation, role-based reviewer access, secure session expiry, secret management, rate limits; no PII in logs. |
| Traceability | Each result linked to input fact IDs, source ID/URL, retrieval/verification time, rule version and criterion trace. |
| Freshness | Show dataset/rule version and last checked/verified time; flag stale/unreviewed data; human review before policy update. |
| Offline resilience | Local cache/outbox, atomic bundle update, retry/backoff, conflict handling, stale-cache warnings. |
| Multilingual quality | Test real supported languages, accents/code mixing; confirm age/income/location; fallback if language unsupported. |
| Maintainability | Rules/data versioned separately from prompts/UI; modular services; typed schemas and API contract. |
| Reproducibility | Docker setup, seed data, env template, automated tests, deterministic test cases, README/runbook. |
| Auditability | Record actor/action/time/case/version; tamper-evident log is future enhancement, not claimed unless implemented. |

# 9. Low-level design (LLD)

## Backend modules

| Module | Responsibilities | Key interfaces |
|---|---|---|
| voice_adapter | Call VerbX STT/TTS/translation; normalize errors; keep transcript + language. | transcribe(audio, language); speak(text, language) |
| conversation_service | Turn/session orchestration; request clarification; switch language without losing facts. | handle_turn(session_id, utterance) |
| profile_service | Validate structured facts, provenance, confirmation state and stable input IDs. | upsert_fact; confirm_fact; get_profile |
| catalogue_service | Jurisdiction-aware candidate retrieval, source metadata, freshness. | search_schemes(profile, intent, cache_version) |
| knowledge_graph_service | Traverse scheme/state/criteria/document/department relations (relational MVP; optional graph DB). | related_schemes; requirements_for_scheme |
| rules_engine | Evaluate typed/versioned rule AST; return criterion outcomes, reason trace, status. | evaluate(profile, scheme_rule_version) |
| document_service | Checklist, optional OCR extraction and document candidate validation. | build_checklist; extract_candidates (stretch) |
| draft_case_service | Draft create/edit, consent, idempotency, session-code handoff and state transitions. | save_draft; handoff; update_status |
| review_service | Role-authorized case lookup, correction request, review decision. | get_case; request_correction; record_review |
| sync_service | Local outbox ingestion, deduplication, version/conflict handling. | sync_events(device_id, events) |
| audit_service | Minimized event log, stable references and correlation IDs. | append_event; query_authorized_audit |
| test_runner | Seed TC fixtures, invoke pipeline, compare expected/actual, save evidence. | run_case(TC1..TC4) |

## Core data entities

| Entity | Fields (illustrative) |
|---|---|
| Scheme | scheme_id, name, state_scope, level, department_id, category, description, benefits, source_id, imported_at, verified_at, review_status, freshness_label |
| SchemeRule | rule_id, scheme_id, version, jurisdiction, structured_conditions, raw_criteria_text, effective dates, review_status, source_id |
| CriterionResult | criterion_id, operator, expected, observed, outcome=PASS/FAIL/UNKNOWN, explanation |
| ProfileFact | fact_id, session_id, attribute, canonical_value, source=voice/text/operator, confidence, confirmed, captured_at |
| DocumentRequirement | doc_id, scheme_id, name, required_for, validity rule if sourced, source_id, review status |
| Draft | draft_id, case_id, version, fields JSON, field_provenance, status, idempotency_key, created/updated_at |
| CaseSession | case_id, session_id, resume_code_hash, expires_at, consent, device_id, sync_state |
| AuditEvent | event_id, actor_id, case_id, action, timestamp, request_id, object_version, minimal details |
| SyncEvent | event_id, device_id, idempotency_key, local_created_at, bundle/rule version, payload, server_result |

## Rule representation

```
{
  "rule_id": "RULE_SCHEME_001_v3",
  "scheme_id": "SCHEME_001",
  "jurisdiction": {"state": "Jharkhand", "level": "State"},
  "all": [
    {"fact": "age", "op": ">=", "value": 60},
    {"fact": "annual_income", "op": "<=", "value": 100000}
  ],
  "evidence_requirements": ["income_certificate"],
```

```
  "source_id": "SRC_001",
  "source_url": "https://official-source.example/rule",
  "verified_at": "YYYY-MM-DD",
  "review_status": "human_reviewed"
}
```

Illustrative only: thresholds and URL above are placeholders, not real scheme policy. Actual rules must come from a reviewed source.

## Evaluation algorithm

- 1) Validate profile schema and normalize canonical values/units.
- 2) Resolve jurisdiction: explicit citizen statement overrides session/kiosk default; if missing, ask. Central schemes remain eligible for consideration.
- 3) Retrieve candidate schemes by state/central scope, category and semantic keywords (vector search optional).
- 4) For each candidate, evaluate each structured criterion using confirmed facts; missing/uncertain values produce UNKNOWN.
- 5) Check proof/document verification state separately from profile criterion pass.
- 6) Derive outcome by policy: failed confirmed criterion → MISMATCH; all conditions pass and proof verified → MATCH; criteria pass but proof missing/stale source → PROVISIONAL; required fact missing → INSUFFICIENT_INFORMATION; ambiguous rule/source → NEEDS_HUMAN_REVIEW.
- 7) Persist trace with input fact IDs, source, rule version and timestamp; produce localized explanation from that trace (LLM may phrase it, not alter it).

# 10. API endpoints and frontend routes

Proposed REST API (reconcile with starter repo/organizer contract before implementation). Prefix: `/api/v1`.

| Method | Endpoint | Purpose |
|---|---|---|
| POST | /sessions | Start citizen/kiosk session; return session ID, expiry and mode. |
| GET | /sessions/{id} | Read session/profile/status permitted to caller. |
| POST | /sessions/{id}/turns | Submit text/transcript; run conversational extraction/clarification. |
| POST | /sessions/{id}/voice/transcribe | VerbX STT adapter (if backend-mediated). |
| POST | /sessions/{id}/voice/speak | VerbX TTS adapter (if backend-mediated). |
| PATCH | /sessions/{id}/profile/facts/{fact_id} | Correct/confirm a fact; trigger re-evaluation. |
| POST | /sessions/{id}/schemes/search | Retrieve candidate schemes by intent/profile/jurisdiction. |
| POST | /sessions/{id}/eligibility/evaluate | Run deterministic rules; return outcomes and trace. |
| GET | /schemes/{scheme_id} | Read scheme, source and freshness metadata. |
| GET | /schemes/{scheme_id}/documents | Read document requirements/checklist. |
| POST | /sessions/{id}/drafts | Create/update draft; require idempotency key. |
| GET | /drafts/{draft_id} | Read draft and field provenance. |
| PATCH | /drafts/{draft_id} | Edit draft with version check. |
| POST | /drafts/{draft_id}/handoff | Consent + create/reuse reviewer handoff code. |
| POST | /reviewer/cases/lookup | Lookup case code after reviewer authentication. |
| GET | /reviewer/cases/{case_id} | Read authorized case. |
| POST | /reviewer/cases/{case_id}/decision | Request correction or record review outcome. |
| GET | /cases/{case_id}/audit | Read authorized audit trail. |
| POST | /sync/events | Upload offline queued events; deduplicate and reconcile. |
| GET | /sync/bundle | Download versioned local scheme/rule bundle. |
| GET | /health | Liveness. |
| GET | /ready | Readiness including DB/dependencies. |
| GET | /admin/schemes/freshness | Review freshness and data-quality status. |
| POST | /admin/schemes/import | Import validated scheme data; admin only. |

## Frontend routes

| Route | Screen |
|---|---|
| / | Landing + language selection |
| /citizen | Citizen conversation and profile confirmation |
| /kiosk | Assisted large-control mode (same flow) |
| /citizen/schemes | Scheme Passport results |
| /citizen/documents | Document Guide/checklist |
| /citizen/draft/:id | Review/edit application draft |
| /case/:code | Permitted resume/recovery flow; code is not sole auth for sensitive content |
| /reviewer | Reviewer queue/status filters |
| /reviewer/case/:id | Case review, evidence, correction/review actions |

| Route | Screen |
|---|---|
| /admin/schemes | Catalogue validation, freshness and import |
| /test-runs | TC1–TC4 runner; expected/actual/evidence |

## Common API rules

- Use request/correlation IDs; typed request/response schemas; explicit error codes; rate-limit session-code lookup.
- Draft/handoff/sync writes require idempotency keys; edits use version/ETag/optimistic concurrency.
- Never expose reviewer endpoints without server-side role checks. Use consent scope to control case visibility.
- Do not put private profile facts or raw documents in URLs or logs.

# 11. Status, idempotency, offline sync and data freshness

## Workflow states

| State | Meaning |
|---|---|
| SESSION_ACTIVE | Citizen interaction in progress. |
| PROFILE_NEEDS_CONFIRMATION | Critical extracted facts require confirmation. |
| MATCH_RESULTS_READY | Screening complete with traceable criteria. |
| DRAFT_SAVED | Draft saved locally/cloud-side; not government-submitted. |
| SYNC_PENDING | Local outbox awaits connectivity. |
| SYNCED | Cloud accepted the idempotent event/draft. |
| PENDING_REVIEW | Consented draft available to authorized reviewer. |
| CORRECTION_REQUESTED | Reviewer needs missing/corrected information. |
| REVIEWED | Reviewer recorded an outcome; show outcome and timestamp. |
| SUBMITTED_EXTERNAL | Only after an approved integration returns verifiable submission receipt. |
| FAILED_RETRYABLE | Action failed; safe retry available. |
| CONFLICT | Version conflict; user/reviewer must reconcile. |

## Idempotency and retries

- Use `Idempotency-Key` for draft creation, handoff and sync event ingestion. A repeated key returns the original resource/result.
- Session/resume code maps to an existing case; repeated submit updates that draft under version control, never creates duplicates.
- A timeout is not success. Show pending/failed and reconcile by querying the canonical status.
- Production session codes should be high-entropy or paired with authentication/consent, expire, be rate-limited and stored hashed. A 4-digit code is demo-only, not sufficient protection.

## Freshness and policy updates

- Every scheme/rule records imported/retrieved/verified dates, source, rule version, review status and jurisdiction.
- Do not hardcode a universal “30 days means stale” threshold unless policy owner defines it. Configure freshness thresholds per source/policy and show exact verified-as-of date.
- Background job may detect source changes, but it must queue proposed rule changes for human verification; crawler/LLM does not auto-publish legal eligibility rules.
- Official reviewer/steward can flag local changes; record who, what source/order, when, and require authorized validation.
- Offline kiosk results retain the bundle/rule version used and are rechecked against current cloud rules before any permitted handoff.

# 12. Security, privacy and trust

- **Consent:** explicitly explain what profile facts/documents will be shared with a reviewer; obtain consent before handoff.
- **Data minimization:** collect only fields needed for selected scheme checks; avoid Aadhaar number, real OTP/passwords, caste/medical details unless necessary, authorized and protected.
- **Shared kiosk:** clear local visible state on exit; expire sessions; separate operator identity from citizen; no sensitive information exposed by code alone.
- **Voice:** disclose external VerbX processing as applicable; do not retain raw audio unless required and consented; use text fallback on failure.
- **Documents:** use synthetic/test documents for demo. OCR can extract candidate data, not certify authenticity, issuing authority or legal validity.
- **Access:** reviewer/admin role checks server-side; seeded demo identities; least privilege and audit of case reads/decisions.
- **Prompt/data safety:** treat retrieved scheme text and uploaded files as untrusted; source content cannot override tool/authorization/rule policy.
- **Audit:** log fact IDs, rule/source versions and actions; avoid raw sensitive values. Tamper-evident/cryptographic audit is a stretch feature, not a claim unless implemented.
- **Human approval:** application draft is reviewed by citizen; any irreversible external action requires explicit approval and real integration confirmation.
- **Truthful UI:** “potential/provisional match” and “draft saved” are not “approved”, “benefit granted” or “submitted”.

## Data freshness risk

Stale criteria can waste a citizen’s travel, fees and effort or cause rejection. Anvaya should make freshness visible and avoid asserting policy currency when it cannot establish it. The screenshot proposed a freshness/confidence score and periodic portal checks; for the MVP, implement source timestamp + review label + warning. A production crawler, change detection and human-approved publication are later work.

# 13. Originality ideas: what to implement vs defer

| Idea from brainstorming/screenshots | Decision | Practical treatment |
|---|---|---|
| Hybrid deterministic/probabilistic eligibility | CORE | LLM extracts/clarifies; deterministic rules evaluate; criterion trace and provisional/unknown statuses. |
| Offline-resilient edge kiosk | CORE DIFFERENTIATOR | SQLite cache + local rules + outbox sync demo; narrow reviewed scheme subset. |
| Scheme Passport (multi-scheme household profile) | CORE | One profile, multiple scheme cards, reasons, gaps, sources and next steps. |
| Document Gap Guide + auto-draft | CORE / OCR optional | Checklist and draft from confirmed facts; optional OCR extracts candidate fields only. |
| Dialect-native/code-mixed semantic layer | SMALL STRETCH | Glossary/translation memory for a few local terms; no fine-tuning unless proven needed. |
| Freshness/confidence + human-in-loop policy review | CORE LITE / production expansion | Display source/version/verified date; admin flags; human-approved rule updates. |
| Kiosk-to-official handoff + retry safety | CORE | Expiring demo code, reviewer queue, idempotent draft, visible statuses. |
| Immutable/tamper-evident kiosk audit | PRODUCTION/STRETCH | Start with append-only event log; cryptographic tamper evidence only if time/security design permits. |
| Proactive seasonal nudges based on weather/agri calendars | DEFER | Potential future opt-in alerts; requires trusted event data, consent, scheduling and safeguards. |
| “Blind kiosk” zero-knowledge entitlement slip | DEFER/RESEARCH | Privacy concept; real zero-knowledge proof design is not a quick feature. Do not call QR token a ZK proof. |
| Voice-biometric/acoustic entitlement token | DO NOT BUILD FOR HACKATHON | Sensitive biometric processing, spoofing, consent and legal/privacy risks; use expiring resume code or authorized account. |
| Sonic-iconic three-glyph UI | OPTIONAL UX EXPERIMENT | Use clear audio/visual states and accessible controls; do not remove textual alternatives. |
| GraphDB/Neo4j | OPTIONAL ARCHITECTURE EXTENSION | Model graph now; adopt graph store only when query/use case and deployment support justify it. |
| Vector search | OPTIONAL RETRIEVAL | Find candidate schemes by semantic description; never decide eligibility with similarity scores. |

## Strongest hackathon bundle

Scheme Passport + voice correction loop + deterministic reason/source trace + Document Gap cards + offline local pre-screen/sync + fast official handoff + TC1–TC4 runner. This combines civic usefulness, technical depth and demo reliability without trying to ship biometric identity, ZK cryptography, a large autonomous crawler or a multi-agent platform.

# 14. Demo narrative and validation plan

## Stage-ready demo (approximately 3–5 minutes)

- 1. Open kiosk mode; choose Telugu; show connectivity/cache version.
- 2. Citizen describes household. Seed the voice case with an intentionally misheard age/income; Anvaya repeats the interpreted value and asks for confirmation.
- 3. Citizen corrects it; show fact ID/state and eligibility recalculation.
- 4. Scheme Passport presents multiple relevant candidates with source citations, rule excerpts, provisional labels and missing document chips.
- 5. Open Document Guide; show what is required and what remains missing; generate draft with field provenance.
- 6. Disconnect network; run cached pre-screen and save a queued draft; reconnect; sync once.
- 7. Switch to reviewer dashboard; enter the handoff code; inspect case, request correction or record review. Show explicit pending/reviewed state—not fake government approval.
- 8. Run TC1–TC4 from the test panel; show expected/actual and trace evidence.

## Acceptance and regression matrix

| Test | Expected assertion |
|---|---|
| TC1 eligible profile | Scheme card includes exact criterion evidence/source and reproducible result. |
| TC2 same name, different states | Correct jurisdiction selected; state-specific rules/source differ as appropriate. |
| TC3 misheard age | No filtering before confirmation; correction changes input fact and re-evaluation. |
| TC4 missing proof | PROVISIONAL/INSUFFICIENT_INFORMATION, document gap, no benefit promise. |
| Offline cache | No network required for local pre-screen; version/freshness displayed. |
| Reconnect sync retry | Same event/key results in one canonical draft; duplicates absent. |
| Stale bundle | Warning; online re-evaluation required before handoff. |
| Voice service down | Text path remains functional and failure is clear. |
| Unauthorized reviewer | Denied server-side; event logged without exposing case. |
| Expired/guessed code | Rate-limited/expired; sensitive case data not returned. |
| Kiosk exit | Sensitive visible state cleared; retention/consent honored. |

# 15. Build roadmap and deliverables

| Milestone | Build | Definition of done |
|---|---|---|
| M0 — inspect starter | Read organizer README, Flask app, Docker Compose, DB/schema, mock portal/SMS/required contracts. | No assumptions conflict with starter; API/data contract documented. |
| M1 — data foundation | Extract CSV, validate/normalize, source metadata; curate small reviewed ruleset and gold cases. | Import report, stable IDs, rule/source provenance, TC fixtures. |
| M2 — rules + KG model | Implement typed profile, graph-shaped catalogue relationships and deterministic evaluator. | Unit tests for PASS/FAIL/UNKNOWN; structured reasons and citations. |
| M3 — citizen voice/text | VerbX adapter, language, profile extraction, confirmation/correction loop. | Voice/text path works; language switch preserves facts; error fallback. |
| M4 — Scheme Passport + documents | Multi-scheme cards, criteria excerpts, doc gaps, draft preview/save/edit. | Full happy path with provisional truthfulness and provenance. |
| M5 — kiosk edge | SQLite reviewed bundle, offline rule evaluation, outbox, reconnect sync. | Disconnect/reconnect demo with deduped draft. |
| M6 — reviewer | Seeded role-based dashboard, case lookup, correction/review, audit. | Authorized handoff and visible status lifecycle. |
| M7 — official test runner | TC1–TC4 preloaded, expected/actual, evidence and reset. | Judge can reproduce each case with one click. |
| M8 — harden/present | Docker runbook, accessibility, privacy checks, error cases, demo recording. | Another teammate can run demo; limitations are honest. |

## Repository deliverables

- Frontend and backend source; API schema/examples; database migrations/seeds; dataset validation/import script; curated rules + sources.
- Docker Compose and `.env.example` (no secrets); local edge bundle build/update script; sync test fixtures.
- Automated unit/integration tests; TC1–TC4 test-runner; README with setup, supported languages, offline boundaries, data provenance and demo script.
- Architecture diagram and brief judge-facing explanation of deterministic rules, graph relationships, edge/cloud division and limitations.

# 16. Open decisions and explicit assumptions

- **Cloud provider:** user-provided screenshot establishes hybrid edge-cloud concept, not a specific AWS/Azure/GCP service mapping. Choose provider only after checking hackathon credits/access; document actual deployment separately.
- **GraphDB:** Knowledge Graph is part of the conceptual model. Decide whether to run Neo4j/GraphDB after reviewing dataset relationships, expected graph queries, deployment time and operational burden. Do not claim it is deployed until tested.
- **Offline scope:** offline applies to cached catalogue + deterministic pre-screen + local draft/outbox. VerbX cloud speech, live official-source retrieval, heavy OCR and cloud PDF generation may require connectivity.
- **Policy source verification:** identify which dataset fields/URLs are official and which rules have human review. The dataset itself may be scraped/third-party and could be stale.
- **Document requirements:** validity windows (e.g., income certificate age) must be sourced for each scheme; do not hardcode generic validity rules.
- **Authentication:** demo credentials and short codes are for evaluation only. Production requires approved identity/access controls.
- **External application:** official brief's permitted action is pre-fill for review against mock/read-only portal. Real submission and status lookup are out of scope unless organizer supplies an authorized API.
- **Rubric:** verify current organizer scorecard; screenshot content may represent draft/proposed rubric or a separate review rubric.
- **Personal data:** define consent, retention, kiosk cleanup and who can access cases before using real citizen data.

## Judge pitch (concise)

"Anvaya turns a citizen's voice into confirmed facts, checks those facts against sourced and versioned welfare rules—not an LLM guess—then explains potential matches and missing documents and prepares a reviewable draft. Its kiosk can perform a limited preliminary check offline and sync safely when connectivity returns. Every result is traceable, provisional where evidence is missing, and ready for human review."

End of specification · Version 1.0